Aula 17 - Estrutura Web (HTML5)

Descrição: Vamos trabalhar com a criação de páginas estáticas para Listagem e Cadastro. Conectar os atributos do formulário (name="") com os conceitos de chaves de Dicionários e atributos de Objetos, mostrando que a interface é apenas uma ponte para alimentar as classes.

Cenário Real:

No desenvolvimento de software profissional, os sistemas que operam **"por trás dos panos"** (Back-End) dependem de uma estrutura sólida **"na frente"** (Front-End) para interagir com o usuário. Sistemas corporativos modernos, como os de gestão de inventário ou controle financeiro, não abrem várias janelas soltas. Em vez disso, a tendência é integrar a **listagem de dados** e o **formulário de cadastro** em uma única tela dinâmica. Essa integração melhora a fluidez e a eficiência da experiência do usuário. Toda essa mágica da interação começa com um esqueleto sólido e bem arquitetado usando o **HTML5**, que servirá de ponte para que os dados preenchidos cheguem às nossas Classes e Dicionários no Python.

Pergunta:

Quando vocês preenchem um formulário online, como vocês acham que o sistema sabe exatamente qual dado pertence ao "Nome", ao "E-mail" ou à "Senha" na hora de salvar no banco de dados?

O Esqueleto da Aplicação e Semântica

A. Conceito:

Todo documento HTML5 exige uma **estrutura básica** que atua como o esqueleto da interface, informando ao navegador como interpretar e renderizar o conteúdo. Elementos como `<!DOCTYPE html>`, `<html lang="pt-br">`, `<head>` e `<body>` são os **pilares indispensáveis** desse documento. A tag `<head>` carrega **metadados fundamentais** para configuração (como **charset** e **viewport**), mas invisíveis diretamente ao usuário, enquanto o `<body>` contém **tudo** o que será **visualizado** na tela, incluindo nossos formulários e tabelas.

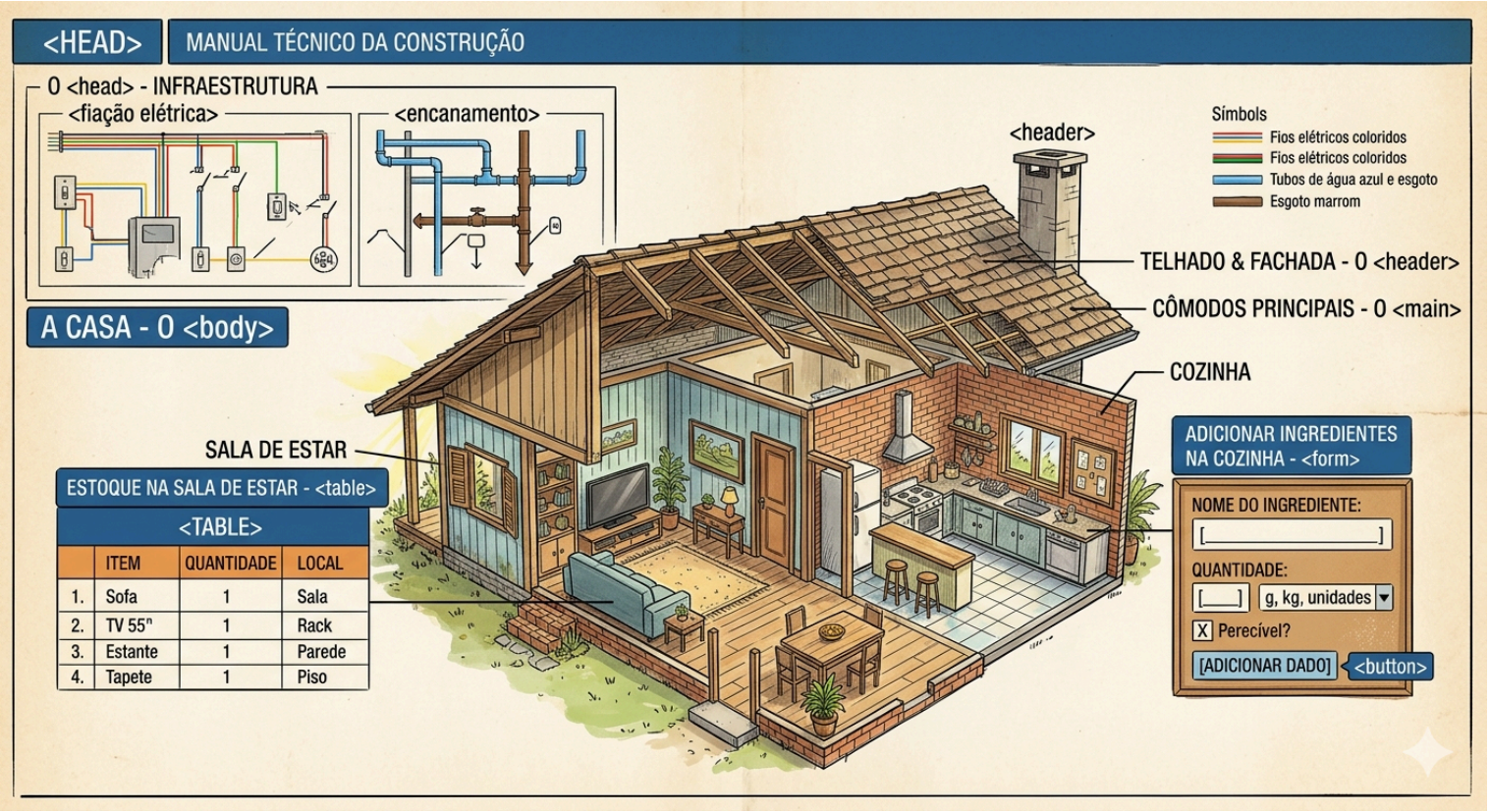
A transição para o HTML5 trouxe a "**Semântica**", o que significa que o código deve ter **significado além da estrutura**. Em vez de agrupamentos genéricos, tags como `<header>`, `<main>` e `<section>` devem ser aplicadas. Isso facilita a manutenção do código para outros desenvolvedores e torna a aplicação legível por **leitores de tela** e robôs de busca (**SEO**).

Dica: Lembre-se que a semântica não é apenas para nós, humanos. Quando você usa `<main>` e `<table>` corretamente, você está ajudando o Google a entender o que é importante no seu site e permitindo que pessoas com deficiência visual naveguem melhor usando leitores de tela.

B. Visualizando:

Imagine a planta de uma casa . O `<head>` é o manual técnico da construção (fiação elétrica, encanamento). O `<body>` é a casa em si, com a `<header>` sendo o telhado, o `<main>` como os

cômodos principais, abrigando uma `<table>` para mostrar o estoque na sala de estar, e um `<form>` na cozinha para adicionar novos ingredientes (dados). *[ver img01.png]*



C. Exemplo de Código:

HTML

```html
<!DOCTYPE html>
<html lang="pt-br">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Sistema de Gestão - Inventário</title>
</head>
<body>
  <main>
    <section id="listagem">
      <h2>Produtos Cadastrados</h2>
      <button>Novo Produto</button>
      <table>
        <thead>
          <tr>
            <th scope="col">ID</th>
            <th scope="col">Nome do Produto</th>
            <th scope="col">Ações</th>
          </tr>
        </thead>
```

```
        <tbody>
          <tr>
            <td>1</td>
            <td>Monitor 24 polegadas</td>
            <td>
              <button>Editar</button>
              <button>Excluir</button>
            </td>
          </tr>
        </tbody>
      </table>
    </section>
  </main>
</body>
</html>
```

D. Explicação linha por linha:

- `<!DOCTYPE html>`: Define a versão do HTML (HTML5) para o navegador.
- `<html lang="pt-br">`: Tag raiz indicando o idioma principal.
- `<head>`: Bloco contendo as configurações de metadados da página.
- `<meta charset="UTF-8">`: Permite o uso de acentuação e caracteres especiais.
- `<body>`: Onde inserimos o conteúdo visível da aplicação.
- `<section>`: Tag semântica que organiza o conteúdo em blocos lógicos.
- `<table>`: Cria a tabela de listagem de dados, essencial para a função "Read" do CRUD.
- `<button>`: Elemento de gatilho que chamará o JavaScript para orquestrar ações futuras.
- `<th scope="col">`: Define um cabeçalho para a coluna, prática importante para acessibilidade.

E. Exercícios Rápidos:

1. Identificação de Papéis (Teórico)

Olhando para o código de exemplo, qual tag você utilizaria para envolver o título "Sistema de Gestão - Inventário" caso ele devesse aparecer no topo da página, acima de todo o conteúdo principal, funcionando como o "telhado" ou fachada da aplicação?

- ( ) `<section>`
- ( ) `<header>`
- ( ) `<footer>`

2. O Detetive do `<head>`

O código atual possui metadados para caracteres e responsividade. Sua tarefa é adicionar uma nova informação técnica ao "manual da construção" (o `<head>`): Adicione uma tag que define o ícone da página (favicon) utilizando o caminho fictício `img/favicon.png`.

3. Expansão da Tabela

A <table> na sala de estar (o estoque) está incompleta. No corpo da tabela (<tbody>), adicione uma segunda linha (<tr>) representando um novo item:

- ID: 2
- Nome do Produto: Teclado Mecânico
- Ações: Dois botões (Editar e Excluir).

4. Organização Semântica

O código utiliza uma <section id="listagem">. Imagine que agora você quer criar uma área separada para o rodapé da página, com os direitos autorais. Onde você deve inserir a tag <footer>: dentro ou fora da tag <main>? Justifique sua resposta com base no conceito de "conteúdo principal".

5. Desafio do Formulário (A Cozinha)

Seguindo sua analogia, falta o formulário para adicionar ingredientes. Abaixo da tabela, mas ainda dentro do <main>, crie a estrutura básica de um <form> que contenha:

1. Um campo de texto (<input type="text">) para o nome do produto.
2. Um botão (<button>) com o texto "Salvar Produto".

O Formulário como Alimentador de Classes

A. Conceito:

O formulário HTML (<form>) não existe isoladamente; ele é o **principal canal de comunicação** entre o **usuário** e o nosso código **Back-End** para operações de **Create** e **Update.** Para que as classes em Python recebam os dados corretamente, o segredo reside no atributo name dos inputs. O valor definido no name ("nome_produto", por exemplo) atuará exatamente como as **chaves** do nosso **Dicionário** ou como os atributos da nossa **Classe**.

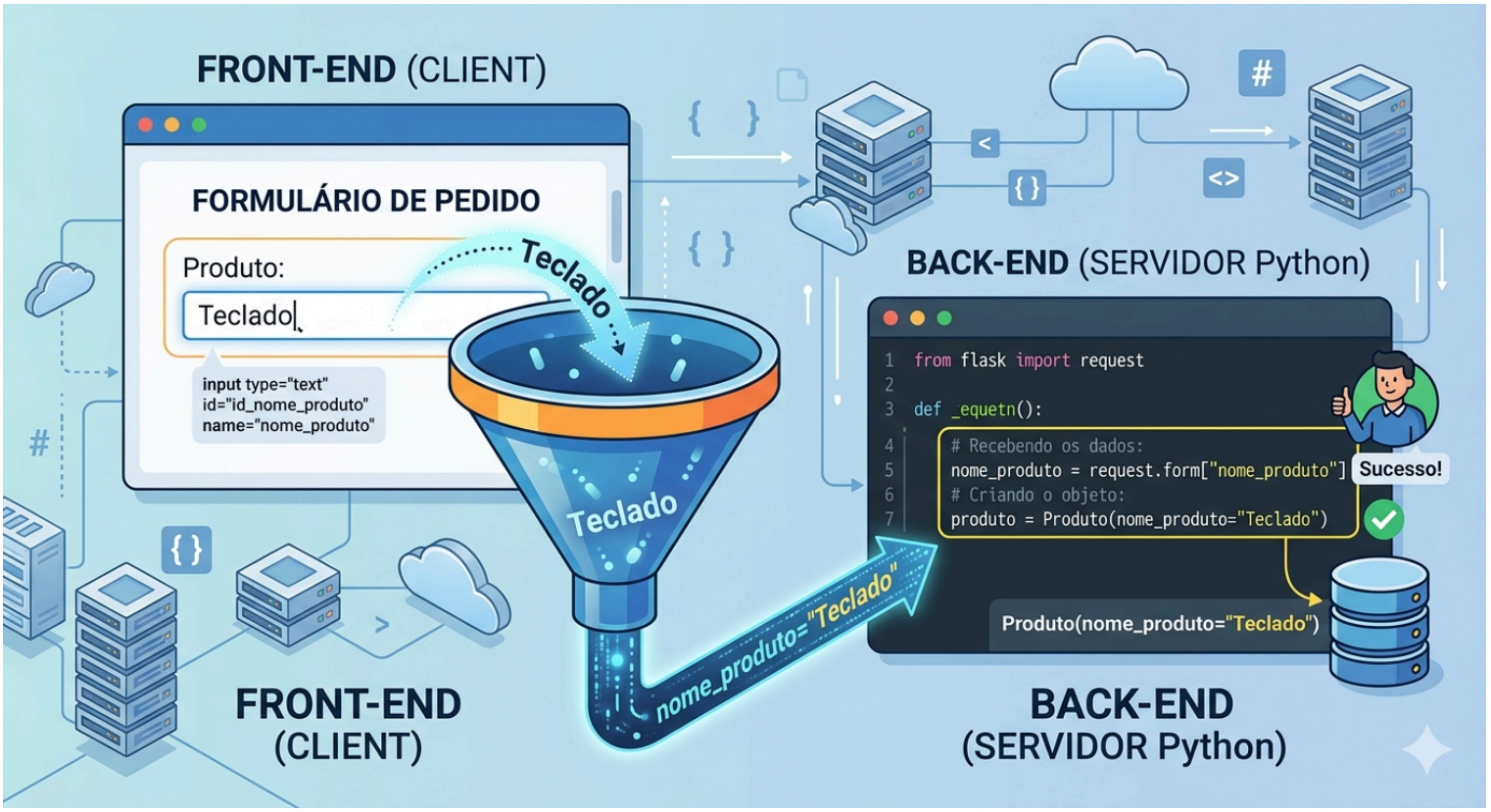
Ao integrar um botão de "**Salvar**" neste formulário, nosso sistema poderá determinar a ação: se houver um campo oculto com um "ID" vazio, é um novo registro; se o campo "ID" estiver preenchido via ação de "Editar", o sistema atualiza um registro existente.

Resumo da Operação:

1. **Usuário preenche:** Digita "Teclado" no campo com name="nome_produto".
2. **Botão Salvar:** Dispara o envio.
3. **Back-End recebe:** O Python lê a "etiqueta" nome_produto e guarda o valor "Teclado".
4. **Verificação de ID:** Se houver um ID, ele sobrescreve o antigo; se não, cria um novo "Teclado" no estoque.

B. Visualizando:

Um funil . O usuário digita "Teclado" em um input que tem o rótulo "Produto" no formulário. Esse input possui name="nome_produto". Esse funil (o envio do formulário) leva essa informação ao Back-End, onde uma variável Python produto = Produto(nome_produto="Teclado") o recepciona perfeitamente.



C. Exemplo de Código:

HTML

```html
<section id="cadastro">
  <h2>Gerenciar Produto</h2>
  <form action="/salvar" method="POST">
    <input type="hidden" name="id" value="">

    <label for="nome">Nome do Produto:</label>
    <input type="text" id="nome" name="nome_produto" required>

    <label for="preco">Preço (R$):</label>
    <input type="number" id="preco" name="valor_venda" step="0.01" required>

    <button type="submit">Salvar</button>
  </form>
</section>
```

D. Explicação linha por linha:

- `<form>`: Estrutura que agrupa elementos de entrada de dados.
- `<input type="hidden" name="id" value="">`: Campo invisível. Se estiver vazio na submissão, cria item; se estiver com ID, atualiza.
- `<label>`: Rótulo semântico associado a um input pelo seu `id` para melhorar a usabilidade.
- `<input type="text" name="nome_produto">`: O atributo `name` obrigatoriamente deve combinar com o nome do atributo da nossa classe em Python.

- `<button type="submit">`: Ao ser clicado, submete todos os `names` e `values` atrelados a este formulário.

E. Exercícios Rápidos:

1. O Elo Perdido (Preenchimento)

No código Python abaixo, o desenvolvedor espera receber o preço de um item. Olhando para o Exemplo de Código acima, qual valor deve preencher a lacuna para que o sistema reconheça o dado?

- Python: `preco_recebido = request.form["_____"]`

2. Criar ou Atualizar? (Lógica)

Um usuário clicou em "Salvar". O Back-End verificou o formulário e encontrou o seguinte:
`<input type="hidden" name="id" value="15">`
Com base no conceito de "Alimentador de Classes", o sistema deve:

- ( ) Criar um novo registro no banco de dados.
- ( ) Atualizar o registro existente que possui o ID 15.

3. Batismo de Atributos (Correção)

Você criou uma classe em Python chamada `Usuario` que possui o atributo `email_cliente`. No seu HTML, o input está assim:
`<input type="email" name="email">`
Pergunta: Por que o Python terá dificuldade em "recepcionar" esse dado automaticamente e qual seria a correção necessária no atributo `name`?

4. A Função do Botão (Múltipla Escolha)

O que acontece exatamente no momento em que o usuário clica em `<button type="submit">Salvar</button>`?

- A) O navegador apenas limpa os campos do formulário.
- B) O navegador agrupa todos os valores dos inputs que possuem o atributo `name` e os envia para a rota definida no `action`.
- C) O navegador salva os dados diretamente no banco de dados, sem precisar do Back-End.

5. Desafio de Escrita (Mão na Massa)

Escreva uma linha de código HTML para um campo de entrada onde o usuário digita a Quantidade do produto. Lembre-se que o nome do atributo na classe Python será `estoque_atual`.

<u>FAQ - Dúvidas Reais:</u>

1. FAQ: Professor, preciso decorar todas as tags do HTML5?

Resposta: Não. É importante entender as principais estruturais (`main`, `section`, `form`, `table`). Com o tempo e o uso diário, elas se tornam naturais. Consulte sempre a documentação.

2. FAQ: Por que o id e o name num <input> frequentemente têm valores parecidos ou iguais?

Resposta: O id é usado no Front-End, por CSS e para associar a tag <label> ao input. O name é a variável despachada para o Back-End ler. Para evitar confusão, muitos adotam o mesmo valor para ambos, embora sirvam para coisas diferentes.

3. FAQ: O que acontece se o <input> estiver fora de uma tag <form>?

Resposta: Ele aparecerá na tela e poderá ser editado, mas ao clicar num botão de "Salvar", os dados daquele input específico não serão empacotados e enviados ao servidor.

4. FAQ: Na vida real, a listagem e o cadastro ficam sempre na mesma página?

Resposta: Para aplicações corporativas e dashboards administrativos (SPA - Single Page Applications), isso é quase um padrão pois agiliza o trabalho do usuário, evitando o recarregamento excessivo de novas páginas.

5. FAQ: Como o JavaScript manipula aquele campo ID oculto que vimos no exemplo?

Resposta: Ao clicar no botão "Editar" de uma linha na <table>, o JavaScript (futuramente) pega o ID daquele produto e o insere automaticamente no campo <input type="hidden"> do nosso <form>. Isso prepara o sistema para um Update.

Exercícios:

Exercício 1: O Esqueleto Fundamental

- **Descrição:** Crie um arquivo chamado exercicio1.html. Neste arquivo, digite a estrutura básica indispensável de um documento HTML5, incluindo as tags <!DOCTYPE html>, <html>, <head> e <body>. No cabeçalho, configure o charset para UTF-8 (para aceitar acentuação) e defina o título da página como "Meu Primeiro Sistema".
- **Entrega:** Arquivo exercicio1.html.

Exercício 2: Construindo a Semântica

- **Descrição:** Em um arquivo exercicio2.html, partindo da estrutura do exercício anterior, organize o corpo (<body>) utilizando as tags semânticas adequadas. Crie um "telhado" ou fachada usando a tag <header>, o conteúdo principal da página usando a tag <main> e defina o rodapé usando a tag <footer>.
- **Entrega:** Arquivo exercicio2.html.

Exercício 3: Listagem de Dados (Tabela Simples)

- **Descrição:** No arquivo exercicio3.html, dentro de uma tag <section id="listagem">, crie uma tabela (<table>) para listar as informações de 3 clientes fictícios. A tabela deve ser dividida entre cabeçalho (<thead>) com as colunas "ID" e "Nome", e o corpo (<tbody>) contendo as linhas (<tr>) com os dados.
- **Entrega:** Arquivo exercicio3.html.

Exercício 4: Acessibilidade na Prática

- **Descrição:** Copie a tabela do exercício anterior para um novo arquivo exercicio4.html. Para ajudar os leitores de tela e robôs de busca (SEO), adicione o atributo scope="col" em todas as tags de cabeçalho (<th>) da sua tabela.
- **Entrega:** Arquivo exercicio4.html.

Exercício 5: O Formulário Base

- **Descrição:** Crie exercicio5.html. Construa uma tag <form> simples para capturar o "E-mail" e a "Senha" de um usuário. Lembre-se de utilizar a tag <label> atrelada ao id do <input> correspondente para melhorar a usabilidade da página.
- **Entrega:** Arquivo exercicio5.html.

Exercício 6: Preparando o Funil para o Python (O Atributo Name)

- **Descrição:** O formulário é a ponte para o Back-End em Python. Em exercicio6.html, crie um formulário de cadastro de um "Paciente". Sabendo que a classe no Python espera os atributos nome_completo, data_nascimento e cpf_paciente, crie três inputs e garanta que o atributo name de cada um seja idêntico a esses atributos da classe.
- **Entrega:** Arquivo exercicio6.html.

Exercício 7: Inputs Específicos e Validação

- **Descrição:** No arquivo exercicio7.html, crie um formulário para cadastrar uma venda. Adicione um campo para o valor da venda (<input type="number">). Para permitir a inserção de centavos (frações numéricas), utilize o atributo step="0.01" e marque o preenchimento como obrigatório usando o atributo required.
- **Entrega:** Arquivo exercicio7.html.

Exercício 8: O Segredo da Atualização (Input Oculto)

- **Descrição:** Crie o arquivo exercicio8.html com um formulário de edição. Para que o Python saiba que a ação é atualizar um registro existente (Update) e não criar um novo, insira um campo invisível na página usando <input type="hidden"> com o name="id". Atribua um valor numérico fictício a ele, como value="10".
- **Entrega:** Arquivo exercicio8.html.

Exercício 9: Ações da Tabela

- **Descrição:** Copie a tabela do Exercício 4 para o arquivo exercicio9.html. Adicione uma terceira coluna no cabeçalho chamada "Ações". Em cada linha de dados (<tr>), adicione a respectiva célula (<td>) contendo dois elementos <button>: um com o texto "Editar" e outro com "Excluir".
- **Entrega:** Arquivo exercicio9.html.

Exercício 10: Integração Single Page (Listagem e Cadastro)

- **Descrição:** Para consolidar tudo em exercicio10.html, simule a interface real de uma aplicação corporativa moderna (SPA). Dentro do <main>, coloque duas seções: a primeira (<section

id="listagem">) deve conter a tabela desenvolvida no Exercício 9. Imediatamente abaixo, a segunda (<section id="cadastro">) deve conter um formulário completo. O formulário deve ter um botão <button type="submit">Salvar</button> para despachar as informações.

- **Entrega:** Arquivo exercicio10.html (o arquivo mais completo da aula).

Como usaremos na Situação de Aprendizagem:

Hoje iniciamos a fundação da interface visual do sistema que vocês estão construindo. A estrutura index.html criada hoje e formatada através de tabela e formulários será a interface real do usuário no projeto final.

Se o HTML estiver bem semântico e os atributos name estiverem idênticos aos atributos das suas classes em Python que já codificamos nas aulas anteriores, a integração com o Banco de Dados será quase automática quando ligarmos o Back-End.

Conexão com a Próxima Aula:

Na próxima etapa do nosso cronograma, avançaremos para o bloco Web Framework! Na **Aula 18**, pegaremos esse HTML estático e daremos vida a ele criando o estilo visual com o CSS adicionando cores e bordas nas tabelas.